Kernel Structure Design for Data-Driven Probabilistic Load Flow Studies

Jingyuan Liu, *Student Member, IEEE*, and Pirathayini Srikantha, *Member, IEEE*

***Abstract*—In power system analysis, probabilistic load flow (PLF) accounts for uncertainties stemming from power entities such as renewable generation systems and consumer demands. In this paper, we present a Gaussian Process (GP) emulator for enabling data-driven PLF on practical distribution networks (DNs) without requiring knowledge of underlying system parameters. The main novelty of our proposal lies in the kernel design process. An ideal kernel allows for greater efficiency during training and inferencing stages without being subject to common issues that include overfitting to the training dataset and poor accuracies. In this proposal, the kernel selection process is formulated as a bi-level optimization problem. This is a very difficult problem to solve as it is composed of discrete variables and non-convex constraints. We overcome these issues by proposing a best-response strategy refinement process to identify an efficient kernel configuration in an iterative manner. The convergence properties of this iterative algorithm and the approximating capabilities of the resulting GP emulator are established using potential game theoretic constructs, universal approximation theorem and representer theorem. The performance of the proposed emulator is then showcased via comprehensive simulations conducted on practical DNs and comparisons with the state-of-the-art.**

***Index Terms*—Artificial intelligence, optimization, power systems.**

I. Introduction

INCREASING climate change concerns and sustainable energy consumption initiatives have led to the proliferation of distributed generation sources such as renewable generation, storage systems, electric vehicles and smart consumer appliances in the modern distribution networks (DNs). While these power entities allow for the reduction of carbon emissions, these have also introduced significant uncertainties in the DN which can lead to operational inefficiencies and outages [1]. Probabilistic load flow (PLF) studies allow system operators to assess the impact of these uncertainties on the operational conditions of the DN [2]. In this paper, we propose data-driven PLF where the underlying model representing the PLF processes is selected in an automated manner via bi-level optimization and game theoretic constructs. The main drawbacks associated with state-of-the-art in PLF are: 1) High computational complexities; 2) Simplifying assumptions; 3) Requirement of internal system parameters for computations; and 4) Extensive manual fine-tuning of PLF model parameters. These are overcome in our proposal as follows.

Traditional PLF studies are based on *Monte Carlo Simulation* (MCS) based techniques. Some examples of existing work based on MCS include techniques based on deep learning [3], approximate Bayesian computations [4], Latin hypercubes [5] and adaptive importance [6]. These proposals do not require any simplifying assumptions. However, the accuracy of these methods is determined by the number of samples taken. As such, techniques in this category are typically associated with high computational overheads. This is not the case with our proposal as once the model is trained, complexity of inferencing is linear with respect to the number of buses as presented in the results section.

Other PLF studies are based on *analytical methods* which include probabilistic load integration [7], multidimensional holomorphic embedding [8] and mixed Gaussian load modeling [9]. To allow for tractable computations, these algorithms typically employ various mathematical approximations and assumptions regarding the electrical system (e.g., linear load flow relations, uncorrelated uncertainty sources) and these can lead to inaccuracies in practical systems [10]. With our proposal, no limiting assumptions are necessary. To demonstrate this, studies presented later in this paper include unbalanced DNs.

Surrogate-based methods are another class of PLF methods where statistically equivalent analytical models that capture the PLF relations are constructed. Non-intrusive low-rank approximation [11], decision tree [12], cumulant tensor [13], polynomial chaos [14] and adaptive kernel estimator [15] are some examples from the literature. Although these techniques require less training data, these also make simplifying assumptions regarding the system model and require significant parameter tuning for calibrating in-built heuristics. In our proposal, parameter-tuning is automated and there is no need for manual customizations.

The final class of proposals are distinguished by the amount of prior information regarding the electrical grid (e.g., line parameters, topology, etc.) required for constructing the PLF models. As such, [3], [4], [16], [17] are parameter-dependent as these incorporate knowledge of DN parameters in the proposed algorithms. Parameter independent methods such

Manuscript received September 13, 2021; revised January 20, 2022; accepted March 5, 2022. Date of publication March 15, 2022; date of current version June 21, 2022. This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC). Paper no. TSG-01470-2021. *(Corresponding author: Pirathayini Srikantha.)*

The authors are with the Department of Electrical Engineering and Computer Science, York University, Toronto, ON M3J 1P3, Canada (e-mail: jliu2325@eecs.yorku.ca; psrikan@yorku.ca).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TSG.2022.3159579.

Digital Object Identifier 10.1109/TSG.2022.3159579

as [12], [18], [19] do not rely on internal information pertaining to the DN. These are however associated with limitations due to restrictions placed on training datasets and load types accommodated by the DN. With our proposal, no such prior knowledge or restrictions are imposed.

As such, the main contributions of this paper are six-fold: 1) Model selection and parameter tuning are automated (i.e., no need for manual customizations); 2) Internal parameters of the DN (e.g., line admittances and DN topology) will not be required during the training and inferencing stages; 3) Simplifying assumptions regarding the physical attributes of practical DNs (e.g., phase imbalance, network topology, etc.) and probability distributions of random variables (e.g., power and voltage) are not necessary; 4) Training datasets for this approach are readily available as large repositories of measurement and state estimation data are compiled over time by supervisory control and data acquisition (SCADA) systems and these are stored locally by utility companies or made available publicly [20]; 5) Once training is complete, the inferencing process entails low computational complexities and does not require complex solvers as would be typically necessary in a model-driven approach; and 6) Training can be continued indefinitely in an offline manner by leveraging on the abundance of computational resources readily available (e.g., cloud systems) to accommodate changing DN conditions.

As such, in this paper, we construct a Gaussian Process (GP) emulator to effectively capture the stochastic attributes of the modern DN. The main contribution of this paper lies in the efficient selection of the kernel used in the GP emulator. This is a difficult problem and is typically overcome by either involving arduous manual parameter tuning or using common kernels that do not necessarily capture important interactions of the system under consideration. One such example is [18] where a GP emulator has been proposed for conducting PLF. In this approach, the authors have manually selected the squared-exponential (SE) local kernel. Main issues associated with local kernels are that these do not extrapolate well [21] and are subject to the curse of dimensionality where the performance degrades as the feature vector increases in size. To overcome these issues, we utilize the notion of aggregate kernels which was first introduced in [22]. These kernels have been demonstrated to extrapolate well and allow for tractable computations even when the feature space has high dimensions. However, for this, the underlying structure selected for the aggregate kernel is important. Manual selection involves heavy parameter tuning and is not tractable. We automate this process by proposing a novel approach for designing an efficient aggregate kernel for the GP emulator which is customized specifically for DN under consideration based on the training dataset without requiring prior knowledge of DN parameters. For this, we formulate the model selection and training problem as a bi-level optimization problem. Then, we leverage upon game theoretic principles to iteratively identify an efficient aggregate kernel structure in a tractable manner that is theoretically guaranteed to converge and approximate the underlying DN dataset well. The performance is showcased via practical simulation and comparative studies.

The remainder of this paper is organized as follows. Section II. introduces the system settings, notations utilized in this proposal and provides a comprehensive background on GP emulators. The proposed kernel design method is introduced and analyzed in detail in Section III. Section IV presents validation studies conducted via comprehensive simulations on practical DNs. The paper is concluded in Section V.

II. System Model and Background

In this section, the assumptions along with the network and load models utilized in this paper are presented. Then, a brief background on GP emulators is provided.

A. Assumptions

In this proposal, we do not make any assumptions pertaining to the nature of loads and DGs connecting to the DN, number of phases, balanced nature of lines/loads, availability of line parameters of the DN, probability distribution of measurement noise, power demands and generation. Thus, our proposal is designed to cater to practical DN systems. The main assumptions that we make in this paper are pertaining to the availability of training datasets.

Due to the widespread integration of advanced metering infrastructure (AMI) and SCADA in the DNs [20], [23], it is assumed that records of net power injections and corresponding states (e.g., bus voltage magnitude, phase angles, etc.) of each phase associated with every DN bus are readily available. Public datasets such as the ones presented in [24] can be utilized to supplement these internal datasets when the availability of these is limited. Another assumption that we make is regarding the components of the feature vector of the GP emulator. The input feature vector is composed of the net power injection into every bus in the DN. This captures the overall power generation in each bus from distributed resources (e.g., renewables, storage, etc.) subtracted by the overall demand in the system. The stochasticity of demand and generation in the DN will be captured in a combined manner in these input values. Traditional load flow analysis methods (i.e., [25]) proposed in the literature also treat net power injections in each bus as inputs. As for the output from the GP emulator, we set it to be a single DN state entity for simplifying the notations presented in the paper. This can be easily extended without loss of generality to multiple state entities as demonstrated in Section IV and in [26].

B. Network, Load and Generation Models

The proposed PLF technique imposes no restriction on the attributes of the electrical system and the connecting power entities. These details are implicitly embedded within the input and output pairs forming the training and testing datasets. These datasets can be obtained from data repositories maintained by utility companies. When these are not available, simulation tools such as MatPower and OpenDSS can be utilized. In this paper, the later method to generate the training and testing datasets. Specifically, MatPower is used to generate the dataset for the balanced electrical system (i.e., IEEE 33-bus system) and OpenDSS to generate the dataset for the unbalanced electrical systems (i.e., 37-bus and 123-bus systems).

The networks models utilized by these simulation tools are based on the following power balance relations:

$$S_r = P_r + \mathbf{j}Q_r = V_r \sum_{j=r}^{n} Y_{mr} V_m^H \ \forall \ r \in N$$

where S_r, P_r and Q_r denote complex, real and reactive power injected by bus r in the system. The set of buses in the DN is denoted by N and the number of buses is denoted by n (i.e., $|N| = n$). V_r and V_m are the voltage phasors associated with buses r and m, Y_{mr} is the line admittance matrix pertaining to the power line (m, r) connecting buses m and r. Each bus can have up to three phases. The dimensions of power, voltage and line admittance depend on the number of phases associated with the buses. Furthermore, the electrical system under consideration can be unbalanced (this is captured by the line admittance matrix). As such, the system parameters (e.g., line admittance matrix, average loads, etc.) are obtained from [24] for the IEEE 33-bus, 37 bus and 123 bus systems.

Next, the load and generation models that govern the cumulative complex power injection at each bus are detailed. In order to assess the effectiveness of the proposed PLF, we consider active DNs in residential areas with significant penetration of diverse consumer demands, electric vehicles and photovoltaic generation units. According to [27] and [28], these consumer entities are spread across the DN and hence can be modelled via normal distributions (also adopted by existing work such as [18], [29], [30] and [31]). We also adopt this approach and the specific parameters of the normal distributions utilized to model consumer loads, electric vehicles and photovoltaics are presented next. The mean values are expressed as a percentage of original demands at each phase of buses corresponding to the DN under consideration.

Complex power injection $_r$ at bus r is composed of three components: cumulative consumer loads, electric vehicles (EVs), and photovoltaic DGs. The consumer loads excluding EVs are modelled as normal distributions whose means and standard deviations are selected to be 125% and 5% of the original loads respectively. The mean values selected aim to encapsulate diversity in power demands. Moreover, the variance values selected are also adopted in the state-of-the-art in PLF such as [18]. Next, the consumer loads caused by the EVs are modelled as normally distributed random variables whose means and standard deviations are set to be 75% and 5% of the original loads respectively. We consider higher penetration of EVs in comparison to existing literature (e.g., [32] and [33] select penetration rate of 10% to 20%) so that greater variability can be introduced in power demands. Furthermore, existing proposals such as [29] have also set the standard deviations of EV loads as 5%. Finally, the power injections from photovoltaics are modelled as normal distributions whose means and standard deviations are set as 100% and 20% of the original demands respectively. The means selected allow the power generated by the PVs to supply all of the original demands in the DN on average. The standard deviations selected have also been adopted in [31].

C. GP Emulator

Next, fundamental definitions of constructs pertaining to GP emulators are presented. GP emulators aim to produce surrogate models of real world systems that are associated with uncertainties. As such, in this paper, input feature vector x into the GP emulator represent specific realizations of the inherently stochastic real and reactive power injections of all phases associated with the DN buses. Hence, an instance i of input x will be referred to as x^i and takes values in d_i dimensional real space (i.e., $\mathbb{R}^{d_i}$). If we denote the set of phases for bus j by ϕ_j, then $d_i = \sum_{j=1}^{n} 2|\phi_j|$ as both real and reactive power injections are accounted for in each bus. The inputs that form the training dataset of size n_X which is used to train the GP are represented by the matrix $X \in \mathbb{R}^{n_X \times d_i}$ where each row is a specific input that contains a snapshot of the real/reactive power injections in the DN at a specific time instance. The system output for the input x^i is denoted as $f(x^i)$ and the collection of all system outputs corresponding to X is denoted as $f(X) = [f(x^1); \cdots ; f(x^{n_X})]$.

Next, the predictive distribution of input x is defined as a GP and its realization is denoted as $g(x)$. The predictive posterior distribution $p(g(x)|f(X), X, \mu(.), k(., .))$ is defined to be:

$$\begin{aligned} &p(g(x)|f(X), X, \mu(.), k(., .)) = \mathcal{N}(m(x), \Sigma(x)) \\ &m(x) = \mu(x) + k(x, X)k(X, X)^{-1}(f(X) - \mu(X)) \\ &\Sigma(x) = k(x, x) - k(x, X)k(X, X)^{-1}k(X, x) \end{aligned} \quad (1)$$

where μ and k are the prior mean and covariance functions for the emulator respectively. Specific components of these functions listed in the above are:

$$\mu(X) = \begin{bmatrix} \mu(x^1) \\ \vdots \\ \mu(x^{n_X}) \end{bmatrix}, \quad k(X, X) = \begin{bmatrix} k(x^1, x^1) & \dots & k(x^1, x^{n_X}) \\ \vdots & \ddots & \vdots \\ k(x^{n_X}, x^1) & \dots & k(x^{n_X}, x^{n_X}) \end{bmatrix}$$

$$k(x, X) = \begin{bmatrix} k(x, x^1), \dots, k(x, x^{n_X}) \end{bmatrix}, \quad k(X, x) = \begin{bmatrix} k(x^1, x) \\ \vdots \\ k(x^{n_X}, x) \end{bmatrix}$$

μ and k functions represent prior knowledge of the load flow relations which are selected by the data engineer. The predictive posterior distribution is the distribution of $g(x)$ for a GP emulator with prior mean and covariance functions μ and k and is trained on $(X, f(X))$. The predictive posterior mean $m(x)$ approximates $f(.)$ and the predictive posterior covariance $\Sigma(x)$ reflects the precision of the such approximation.

Next, we detail the prior mean and covariance functions adopted in this proposal. As no prior information of the underlying physical model is available, in this proposal, we adopt the zero function as the prior mean function for GPs. As the GP emulator with zero prior mean has been demonstrated to be flexible and is able to model the predictive mean well during the training process, this choice of prior mean is acceptable [34]. Thus, the accuracy of predictions are highly dependent on the prior covariance function. The prior covariance for a GP is typically represented by a kernel function k that is associated with a set of hyper-parameters θ which are inferred during the training process. As the performance of the GP emulator relies heavily on the prior covariance, careful selection and design of the kernel function and associated hyper-parameters will be necessary. One example is the SE kernel defined as $k(x, x') = \sigma_f^2 exp(-\frac{\|x-x'\|^2}{2l^2})$ which was utilized in [18] to conduct PLF. $\|.\|$ denotes the L_2 norm. The

TABLE I
COMMONLY USED KERNELS FOR GP LEARNING

Kernel Name	Kernel Function Formulation
Rational Quadratic	$\sigma^2 \left(1 + \frac{\|\|x-x'\|\|^2}{2\alpha\ell^2}\right)^{-\alpha}$
Periodic	$\sigma^2 \exp\left(-\frac{2\sin^2(\pi\|\|x-x'\|\|/p)}{\ell^2}\right)$
Linear	$\sigma_b^2 + \sigma_v^2(x-c)(x'-c)$

hyper parameters associated with the SE kernel are σ_f and l which are the signal variance and length-scale respectively. Some other examples of kernel functions are listed in Table I.

In this proposal, we utilize *additive* kernels which is detailed in Section II-D. Once the kernel function has been selected, hyper-parameters must be computed from the training dataset. For this, the Maximum Likelihood Estimation (MLE) method is commonly employed. MLE aims to compute the optimal hyper-parameters that maximize the marginal likelihood function over the training dataset. This method has been adopted in the state-of-the-art in GP emulator construction for PLF [18]. The marginal likelihood function for a zero-mean GP model $\mathcal{GP}(0, k(.,.))$ trained on the dataset $(X, f(X))$ is:

$$\begin{aligned} L(k(.,.)|X, f(X)) &= p(f(X)|X, k(.,.)) \\ &= \mathcal{N}(f(X)|0, k(X,X)) \\ &= (2\pi)^{-\frac{n_X}{2}} \times [det(k(X,X))]^{-\frac{1}{2}} \\ &\quad \times exp\left\{-\frac{1}{2} f(X)^T k(X,X)^{-1} f(X)\right\} \end{aligned} \quad (2)$$

where $det(.)$ denotes the determinant of a matrix. From (2), the marginal likelihood is defined as the conditional probability for the emulator output to be equal to $f(X)$ given the input matrix X and the kernel $k(.,.)$. Thus, higher the value of L, the more accurate the prediction by the GP emulator. Moreover, L is a balanced metric that is used widely for computing kernel parameters. This is associated with an intuitive interpretation where $[det(k(X,X))]^{-\frac{1}{2}}$ in L controls the representative capacity of the model and $exp\{-\frac{1}{2} f(X)^T k(X,X)^{-1} f(X)\}$ evaluates model's fit with data [35]. Thus, MLE based training will lead to balanced kernels that are capable of representing complicated relations such as those present in PLF studies.

D. Additive Kernels

We utilize additive kernels to construct the kernel k. With additive kernels, simple kernels such as those presented in Table I are combined together via summations and products so that complex interactions in the input space can be captured closely. These simpler kernels are referred to as base kernels. In studies conducted in this paper, the base kernel is selected to be the SE kernel. The base kernel alone captures local dependencies via the L-2 norm operation. However, this approach does not allow for effective extrapolation as only local interactions are accounted for. As such, first, the m^{th} order kernel $k_{add_m}(x, x')$ is defined as follows:

$$k_{add_m}(x, x') = \sigma_m^2 \sum_{\mathscr{C} \in \mathscr{C}_{d_i}^m} \left[\Pi_{r \in \mathscr{C}} k_r(x_r, x_r')\right] \quad (3)$$

where x and x' represent the two inputs into k_{add_m} with dimension d_i. The set $\mathscr{C}_{d_i}^m$ represents all possible m digit combinations of integers in the range of 1 to d_i. $k_r(x_r, x_r')$ is the base kernel. Thus, the m^{th} order kernel is a summation of the all combinations of products of m base kernels. When m is 0, the kernel k_{add_0} is set to be the base kernel itself. These m^{th} order kernels are combined together as follows to form the final kernel k:

$$k(x, x') = \sum_{s_m \in S} s_m k_{add_m}(x, x') \quad (4)$$

where $s_m \in \{0, 1\}$ indicates the inclusion of the additive kernel k_{add_m}. If s_m is set to 1 for all m, then the kernel is a highly complex entity that can capture intricate interdependencies in the training dataset. However, this can result in over-fitting and the kernel will not generalize well for new datasets. On the other hand, if $s_m = 0$ for most m values, then this will result in a simpler representation of the interdependencies in the dataset and will not extrapolate well. Thus, the specific values selected for s_m determine the structure of the final kernel and ultimately the performance of the GP emulator. Our aim is to compute the most efficient structure of the final kernel by determining ideal configurations for s_m for all $1 \leq m \leq d_i$. We formalize this problem and the proposed algorithm next.

III. PROPOSED KERNEL DESIGN METHODOLOGY

In this section, the kernel design problem is formally presented as a bi-level optimization problem. The proposed iterative algorithm is detailed along with theoretical studies on convergence and approximation capabilities of the resulting GP emulator. Then, a discussion on extending the kernel design methodology to surrogate-based PLF methods is presented.

A. Kernel Design Problem Formulation

The kernel design problem is formulated as a bi-level optimization problem where the optimal hyper-parameters are computed for the additive kernels that are selected to compose the final kernel. We introduce the order set S which is composed of the variables s_m for $1 \leq m \leq d_i$ and the values taken by these. S represents the structure of the final kernel as it contains information on which additive kernel is selected to compose k. Another consideration is the size of the set S. We define the final kernel defined by the order set S and corresponding hyper-parameters as $k_{AK_S}(.,.|S, \theta_S)$.

The input dimension d_i will be large especially when the DN under consideration is composed of a large number of buses. Then, the complexity of the final kernel will become excessively high and lead to overfitting. For this reason, we allow a limit o to be imposed on m where $o \leq d_i$. Hence, the set S now contains o elements. As such, the bi-level optimization problem is composed of $\mathcal{P}_u$ which computes the optimal order set S over the training dataset $(X_u, f(X_u))$ and $\mathcal{P}_l$ which computes the optimal hyper-parameters for the specific order set S over the training dataset $(X_l, f(X_l))$ [36]. Both training datasets do not overlap with one another and are obtained by partitioning the original training dataset $(X, f(X))$ so that over-fitting

issues can be avoided. $\mathcal{P}_u$ selects the optimal order set S whose elements take binary values (i.e., $C1$) and the set of optimal hyper-parameters obtained for each value of $s \in S$ are available in the set $\Theta(S)$ (i.e., $C2$). The objective function of $\mathcal{P}_u$ is the negative MLE as we are aiming to maximize the likelihood of observing the outputs $f(X_u)$ given the inputs X_u for the selected order set S and associated hyper-parameters θ_S. In $\mathcal{P}_l$, the optimal hyper-parameters are computed for a specific order set S over the data set $(X_l, f(X_l))$. The objective of this problem is also the negative MLE.

$$\begin{aligned}
\mathcal{P}_{K_u}\colon \min_{S,\theta_S}\ & F(S,\theta_S) = -L\big(k_{AK_S}(.,.|S,\theta_S)|X_u, f(X_u)\big) \\
s.t.\ & s_m \in \{0,1\}\ \forall\ 1 \le m \le o,\ S = \{s_1,\ldots,s_o\} \quad [C1] \\
& \theta_S \in \Theta(S) \quad [C2] \\
\mathcal{P}_{K_l}\colon \min_{\theta_S}\ & G(S,\theta_S) = -L(k_{AK}(.,.|S,\theta_S)|X_l, f(X_l))
\end{aligned}$$

This bi-level problem is NP-hard due to the non-convexities introduced by the associated variables and constraints. It is thus difficult to solve directly. We propose an iterative algorithm to overcome these difficulties.

B. Proposed Solution and Performance

The notion of potential games is utilized to iteratively compute a solution to the bi-level optimization problem formulated in the above. This iterative optimization process is guaranteed to converge to a local optima. We show in Section IV that the cost computed for the locally optimal solution is very close to the globally optimal value which is computed via brute force techniques. In the next section, we demonstrate that the solution from the proposed algorithm allows for better generalization than the actual optimal solution as the actual optimal solution can lead to overfitting issues.

As a first step to deriving the proposed iterative algorithm, we express the hyper-parameters computed in $\mathcal{P}_{K_l}$ as the mapping $\theta_S = v(S, X_l, f(X_l))$. Then, $\mathcal{P}_{K_u}$ can be equivalently expressed as $\mathcal{P}_K$:

$$\begin{aligned}
\mathcal{P}_K\colon \min_{S}\ & F(S) = -L\big(k_{AK_S}(.,.|S, v(S,X_l,f(X_l)))|X_u, f(X_u)\big) \\
s.t.\ & s_m \in \{0,1\}\ \forall\ 1 \le m \le o,\ S = \{s_1,\ldots,s_o\} \qquad (5)
\end{aligned}$$

In order to iteratively compute a solution to $\mathcal{P}_K$, in the first iteration, the initial value of every element in S is randomly selected from the discrete set $\{0,1\}$. Then, in the subsequent iterations, say t, an element s_m is randomly selected from S. Suppose that the current structure S is denoted as S^t and if the value of s_m is switched from the current value to the other value in the set $\{0,1\}$, the resulting structure is denoted as S^{t+1}. If $F(S^{t+1}) < F(S^t)$, then S is assigned the structure S^{t+1}. Otherwise, S remains as S^t. Then, another element in S selected randomly and the above is repeated again. This is continued until $F(S^t) = F(S^{t+1})$. At every iteration, a search is conducted around the local neighborhood of S candidates. These local searches are guaranteed to converge to the Nash equilibrium which is also the local optimum as shown later in this section. One observation that can be made is that the optimal hyper-parameters are computed for every S^{t+1} by solving $\mathcal{P}_{K_l}$. We show in Section IV that the number of iterations necessary to reach equilibrium is significantly lesser than 2^o (i.e., the total number of combinations possible for S when using a brute force approach). This renders this additional computation an insignificant overhead when compared to the brute force approach. This algorithm is summarized in Algorithm 1.

Algorithm 1: Proposed Kernel Design Algorithm

1 **Initialization**:
- Each element in S^0 is initialized with randomly selected values in $\{0,1\}$.
- The set V_c of visited OS is initialized with the empty set $\emptyset$.

Algorithm:
1) A random OS S is selected from $\hat{\mathcal{S}}_o^c \setminus V_c$ for evaluation.
2) If $F(S) >= F(S^c)$, let $V_c \leftarrow (V_c \cup S)$. Otherwise, $S^c \leftarrow S$ and $V_c \leftarrow \emptyset$.
3) If $V_c = \hat{\mathcal{S}}_{o_c}^n$, then the iterative update process is terminated. Otherwise, return to the first step.

To obtain the convergence properties for the proposed algorithm, $\mathcal{P}_K$ is formulated as an equivalent game $\mathcal{G}_K$. Then, Algorithm 1 is treated as an equivalent game-theoretic decision-making process. The main elements of a game are the set of participating players $\mathcal{P}$, set of strategies available to each player $\mathcal{T}$ and the cost $\mathcal{C}$ which is a function of the strategies selected by the players. In our setup, every element of S represents a player. The strategies available to each player is $\mathcal{T} = \{0,1\}$ and the associated cost is $\mathcal{C} = F$. In a potential game, a player m is randomly selected and a decision as to whether or not to switch from the current strategy is made. All other players do not make any switch during this period. Suppose that the player is contemplating a switch from $a \in \mathcal{T}$ to $b \in \mathcal{T}$ and the strategies selected by all other players are collectively denoted as $\mathcal{T}_{-m}$. If the change in local costs $C_m \in \mathcal{C}$ due to this change is the same as the change in global cost $\mathcal{C}$ as follows:

$$C_m(a,\mathcal{T}_{-m}) - C_m(b,\mathcal{T}_{-m}) = C(a,\mathcal{T}_{-m}) - C(b,\mathcal{T}_{-m}),\ m \in \mathcal{P} \qquad (6)$$

then $\mathcal{G}_K$ is an Exact Potential Game (EPG). This is the case for Algorithm 1 as each player evaluates F which is also a global cost function prior to a strategy switch. With EPG, the stationary point is a local optimum and Nash Equilibrium. Next, we establish that Algorithm 1 results in a stationary point regardless of the initial selection of values in S.

For this, we define *best-response* strategy revision for player m to be the selection of a strategy $a \in \mathcal{T}$ that minimizes the cost C given that all strategies $\mathcal{T}_{-m}$ of other players are fixed. According to [37], if the evolution of costs is according to the following trajectory and the number of strategies available is finite, then convergence to a stationary point will occur in finite time:

$$C_1 > C_2 > \cdots > C_T \qquad (7)$$

where C_t indicates the cost incurred at revision iteration t and $T < \infty$ is the iteration number at which stationarity is attained. Since $\mathcal{G}_K$ is an EPG, this stationary point is also a local optimum and Nash Equilibrium. In Algorithm 1, as strategy revision is also based on best-response and the number

of strategies available to each player is 2 which is finite, it can be concluded that the iterative revisions will converge to the local optima within finite time. The resulting structure is denoted as S^*.

Next, we study how closely the posterior mean resulting from the kernel $k_{AK_{S^*}}$ approximates the load flow relation $f(.)$. The zero-mean GP associated with this kernel and the predictive posterior mean of the GP is denoted as $\mathcal{GP}(0, k_{AK_{S^*}}(.,.))$ and $m_{AK_{S^*}}(.)$ respectively. The supremum norm defined on $\mathbb{R}$ is denoted as $\|.\|_\infty$. From the definition of arbitrary closeness presented in [38], a function $q(.)$ that maps $\mathbb{R}^{d_i}$ to $\mathbb{R}$ can approximate $f(.)$ arbitrarily closely if $\|q(.) - f(.)\|_\infty$ converges to 0 as the number of samples in $(X, f(X))$ tends to infinity. We leverage on the Representer Theorem detailed in [38] and the universal approximating property presented in [39] to establish that the predictive posterior mean of the GP emulator resulting from Algorithm 1 can approximate $f(.)$ arbitrarily closely under certain practical conditions. Details for this proof are presented in the Appendix.

C. Future Extensions

Our proposal can be combined with other surrogate based methods such as polynomial chaos and cumulant tensors (e.g., [13] and [14]) to improve the performance of these. These can be explored as future extensions of this work. In the following, a brief outline of how this integration can take place is provided.

Expansion methods such as polynomial chaos and cumulant tensors approximate the distributions of the output random variables by representing them as polynomial functions of other random variables. In order to improve the computational performance, the current state-of-the-art in expansion based methods such as those introduced in [13] and [14] truncate the polynomial functions by removing their higher order terms. According to [41], this truncation scheme can lead to either large truncation errors or significant computational burdens for high dimensional inputs (i.e., curse of dimensionality). These can be improved by replacing the truncation scheme with a polynomial selection technique that aims to identify the best combination of polynomial terms with pre-selected upper limits on the number of terms and maximum degree of individual terms. Since the search for best polynomial combinations can also be formulated as a combinatorial optimization problem similar to the GP kernel design problem addressed in our proposal, the design approach utilized in our proposal can be adapted for polynomial selection in expansion based surrogate methods.

IV. Results

In this section, the proposed algorithm is evaluated on practical IEEE benchmark systems to demonstrate that the theoretical convergence and approximation guarantees established in the previous section also hold in realistic settings. As such, we consider three DNs that capture various types of practical considerations and these are: 1) Balanced 12.66 kV IEEE 33-bus DN; 2) Unbalanced 4.8 kV IEEE 37-bus DN; and 3) Unbalanced 4.16 kV IEEE 123-bus DN. The specifications of these DNs can be found in [24]. Furthermore, in order to highlight the contributions of our work with respect to the state-of-the-art, we have also included comparative analysis with the Gaussian kernel based PLF proposed in [18] and the optimal kernel structure obtained for the training datasets via brute force search.

A. Generation of Datasets in Active DNs

The manner in which datasets utilized to train and validate the kernels are generated for the three benchmark systems is presented next. The feature vectors X contain the net power injections at each phase of every bus in the DN under consideration. These are stochastic in nature due to the penetration of renewable energy sources and variable consumer demands at each bus (as outlined in Section II-B). As such, the input features in the datasets are composed by sampling from the distributions listed in Section II-B. Then, the corresponding outputs $f(X)$ which are bus voltage magnitudes are generated using MATPOWER and OpenDSS [42]. The total number of points generated for the training and the testing datasets are 400 and 100 respectively. These sizes are comparable to datasets used in other existing work like [43]. A MCS based PLF technique utilizing 10000 datapoints serves as the benchmark for evaluation studies conducted later on in this paper. The resulting dataset is partitioned into training $(X, f(X))$ and testing $(X_t, f(X_t))$ datasets according to the 80/20 ratio [44]. For the proposed kernel design algorithm, we first partition $(X, f(X))$ equally into datasets $(X_l, f(X_l))$ and $(X_u, f(X_u))$. Then, we train the kernel with the structure from Algorithm 1 on $(X, f(X))$ unless otherwise mentioned. During the training phase, the hyper-parameters of the kernels are computed using the GPflow library [45]. We have selected the maximum kernel order o to be 8. Thus, the number of unique kernel structures that can be obtained is 2^8. In this section, we compare the performance of the kernel produced by our algorithm with that associated with the optimal kernel structure computed for the testing dataset. Since the optimal kernel is computed via brute force search, selecting o to be 8 allows for tractable search. This also limits overfitting as typical with overly complex models [44]. We further define the iteration number of the proposed design process as the number of times the proposed process evaluates a kernel.

B. Evaluation Metric

In order to evaluate the performance of the proposed method, three different metric is presented in this section. First, we introduce the difference factor D which evaluates the change in likelihood incurred by the proposed design method. Then, we present the Root Mean Square Error (RMSE) R which measures the emulator output's fit with data. Next, we detail the Kullback-Leibler Divergence (KLD) D_{KL} which quantitatively measures the difference between two random variables.

We first present the formulation of the difference factor D which is defined as follows:

$$D = \frac{log(L(k(.,.)|X_E, f(X_E))) - log(L(k_\omega(.,.)|X_E, f(X_E)))}{log(L(k_\alpha(.,.)|X_E, f(X_E))) - log(L(k_\omega(.,.)|X_E, f(X_E)))} \quad (8)$$

where k is the kernel we aim to evaluate and $(X_E, f(X_E))$ is the dataset k is evaluated on. The first term in the numerator is the log likelihood associated with k. The second terms in both the numerator and the denominator are the same and they are the log likelihood obtained for an GP kernel k_ω whose structure is associated with the worst performance (i.e., least likelihood) for $(X_E, f(X_E))$. The first term in the denominator is the log likelihood for the GP kernel k_α whose structure is associated with the best performance (i.e., highest likelihood) for $(X_E, f(X_E))$. Except for the log likelihood associated with k, all terms in this metric are computed via brute force. This metric allows for normalized insights into the performance of the kernel k with respect to the kernels that are associated with the worst or the best structures for the specific dataset $(X_E, f(X_E))$. As such, D will take values in $[0, 1]$. If the performance of a given kernel is close to the optimal kernel for $(X_E, f(X_E))$, then D will be close to 1. Otherwise, D will be close to 0.

Then, the formulation of the RMSE which is denoted as R is presented as follows:

$$R = \sqrt{\frac{(\|m(X_E) - f(X_E)\|)^2}{n_E^X}} \tag{9}$$

where $m(.)$ is the predictive mean introduced in Section II-C, $(X_E, f(X_E))$ is the dataset the metric is evaluated on, and n_E^X is the number of samples in $(X_E, f(X_E))$. The numerator term in Eq. (9) measures the difference between the predictive mean and the actual output from the emulator constructed with the proposed emulator.

Next, in order to measure the distances (i.e., D_{KL}) between two distributions, we utilize the Kullback Leibler Divergence (KLD) metric that is defined as follows:

$$D_{KL}(\pi_1 \| \pi_2) = \int_{-\infty}^{\infty} \pi_1(r) log\left(\frac{\pi_1(r)}{\pi_2(r)}\right) dr \tag{10}$$

where π_1 and π_2 are the distributions of two random variables. According to [46], $D_{KL}(\pi_1 \| \pi_2)$ can also be interpreted as a measure of the information loss when π_2 is used to approximate π_1.

C. Initial Conditions and Convergence

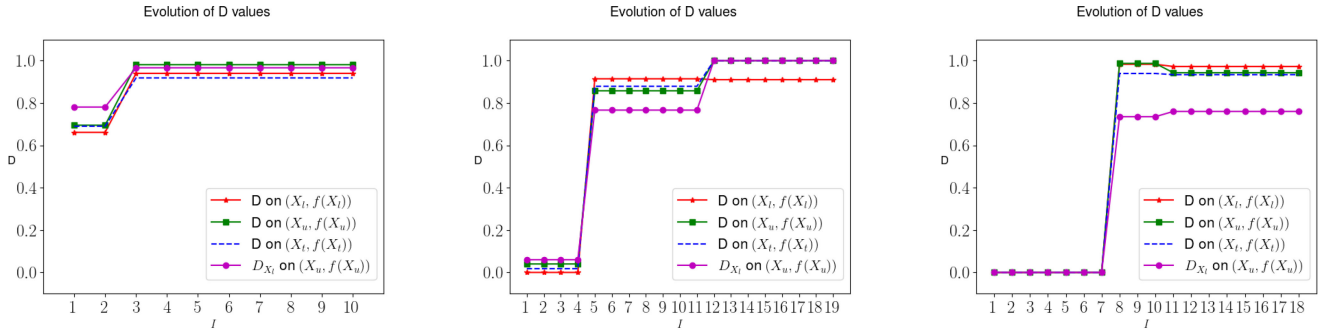
The proposed algorithm is iterative in nature. Constructs from potential games, representer theorem and universal approximation theorem have been leveraged in the previous section to theoretically show that the proposed algorithm is guaranteed to converge and the resulting kernel is able to approximate the distribution of the underlying dataset. This is demonstrated in this section in practical DN settings. First, the impact of the initial kernel structure on the convergence characteristics is studied on all three IEEE benchmark systems. Then, the evolution characteristics of D for the three DNs at every iteration is studied. In all the results presented in this section, kernels are designed to approximate the voltage magnitude of a randomly selected bus and phase in the DN under consideration. As such, for the IEEE 33 bus system, bus 15 is selected (as this is a balanced system there is no need to select a specific phase); for the IEEE 37 bus system, bus 27 and phase C is selected; and for the IEEE 123 bus system, bus 87 and phase B is selected.

TABLE II
IMPACT OF INITIAL KERNEL STRUCTURE ON CONVERGENCE PROPERTIES OF ALGORITHM 1

DN System	$\mu(I_t)$	$\sigma(I_t)$	Dataset	$\mu(D_0)$	$\mu(D_{I_t})$
33-15	21.6	9.89	$(X_l, f(X_l))$	0.823	0.908
			$(X_u, f(X_u))$	0.864	0.932
			$(X_t, f(X_t))$	0.843	0.900
37-21-C	16.6	4.765	$(X_l, f(X_l))$	0.384	0.493
			$(X_u, f(X_u))$	0.352	0.500
			$(X_t, f(X_t))$	0.371	0.503
123-87-B	13.3	2.5407	$(X_l, f(X_l))$	0.685	0.977
			$(X_u, f(X_u))$	0.683	0.975
			$(X_t, f(X_t))$	0.656	0.934

In Table II, the impact of the initial kernel structure on the convergence of the proposed algorithm is studied. For all three DNs, 10 initial kernel structures that abide by the maximum kernel order o are randomly generated. Then, 10 runs of the proposed algorithm are executed where in each run, the initial kernel structure is set to be one of the randomly generated structures. I_t denotes the total number of iterations executed to achieve convergence starting from the initial kernel structure with the proposed algorithm. The first metric in Table II is the average number of iterations $\mu(I_t)$ entailed for the convergence of the algorithm for each one of the 10 kernel structures that are selected as initial conditions. The second metric is the sample standard deviation corresponding to the total number of iterations needed for convergence $\sigma(I_t)$. The third metric $\mu(D_0)$ is the average value of D calculated for the initial kernel structures and the fourth metric $\mu(D_{I_t})$ is the average value of D obtained after convergence.

From Table II, the average number of iterations necessary for convergence (i.e., $\mu(I_t)$) varies for different systems. Nevertheless, these values are much smaller than the total number combinations that must be examined to compute the optimal kernel structure in a brute force manner (i.e., by a factor of 10). Moreover, for all systems examined the standard deviations for convergence iterations are smaller than the averages for convergence iteration by a factor of 2. Thus, the convergence for the proposed design process is consistently fast for all systems and the convergence iteration for various initial kernel structures does not deviate significantly from the average convergence iteration. Next, for all systems the $\mu(D_0)$ values associated with different datasets are close to each other and the $\mu(D_{I_t})$ values associated with different datasets are also similar. Moreover, $\mu(D_{I_t})$ is notably higher than $\mu(D_0)$ for all systems. Thus, for all systems the proposed design process in Algorithm 1 generally leads to notable performance improvement regardless of the initial OS selected. Furthermore, the performance improvements generalizes well for all datasets examined since for all systems Algorithm 1 leads to similar improvements in $\mu(D_{I_t})$ values for different datasets. From the data presented for all systems, it can also be observed that the performance of the proposed design process does not decrease for systems from larger DNs.



(a) System 33-15. (b) System 37-21-C. (c) System 123-87-B.

Fig. 1. Convergence study for the proposed algorithm.

TABLE III
RMSE FOR THE PROPOSED ALGORITHM

DN System	Dataset	R (p.u)
33-15	$(X, f(X))$	0.00212
	$(X_t, f(X_t))$	0.00322
37-21-C	$(X, f(X))$	0.00089
	$(X_t, f(X_t))$	0.00071
123-87-B	$(X, f(X))$	0.00031
	$(X_t, f(X_t))$	0.00049

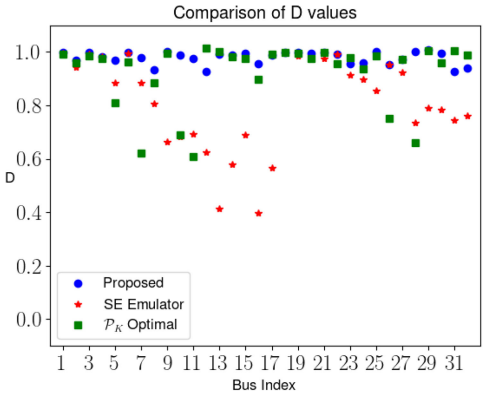


Fig. 2. Comparative study for IEEE-33 DN.

Thus, Algorithm 1 can be easily scaled to systems with high dimensional input vectors.

Next, Figs. 1a–1c plot the D values at every iteration of Algorithm 1 for all systems examined. Here, the difference factors associated with the kernels trained on $(X, f(X))$ and $(X_l, f(X_l))$ are denoted by D and D_{X_l} respectively. For the simulations presented in these figures, the initial kernel structures for Algorithm 1 are represented by zero vector order sets. From these figures, it can be observed that the D_{X_l} values evaluated on $(X_u, f(X_u))$ increase monotonously for all systems. Moreover, the D values associated with all curves belonging to the same test system remain the same at specific iterations while changing in other iterations. This is expected as the proposed iterative algorithm ensures that every change in kernel structure occurs only when the cost for $\mathcal{P}_K$ decreases at that specific iteration by the incumbent order set. Furthermore, these figures show that the performance improvement enabled by Algorithm 1 generalizes well to all datasets examined as there exists no significant divergence among the trends of the D values evaluated on all three datasets.

D. Generalization Analysis

To evaluate the performance of the proposal on training and out-of-sample data points, the RMSE is evaluated on both the training and testing (i.e., out-of-sample) datasets for the three DN systems (i.e., IEEE 33, 37 and 123 bus systems). These results are tabulated in Table III. It is clear from these results that the RMSE values are close to one another for the training and testing datasets for each one of the DN systems considered. This implies that the proposed algorithm is able to design a kernel that generalizes well. Furthermore, the RMSE values are below 10^{-3} p.u. which indicates that the outputs from the emulator are close to the corresponding predictive means.

E. Comparative Study

To validate the performance of the proposed method against the state-of-the-art, we conduct comparative studies of the proposed method against the SE kernel emulator based PLF introduced in [18] and the traditional MCS based PLF method commonly used as a benchmark. First, the D values associated with both PLF methods are compared against those associated with the optimal solution for $\mathcal{P}_K$ for every testing dataset associated with the buses in the IEEE-33 DN. Then, the distributions of the output random variable obtained from the proposed and the SE based PLF methods are compared against those obtained from the benchmark MCS based PLF method. Next, the computational performance of the proposed method is compared against that of the MCS method.

In Fig. 2, the D values computed for the testing datasets associated with all buses in the IEEE-33 DN are plotted for the three PLF methods considered. In this figure, the PLF methods associated with the kernel resulting from the proposed design process, the kernel utilized in [18] and the kernel obtained from the optimal solution for $\mathcal{P}_K$ are referred to as Proposed, SE Emulator and $\mathcal{P}_K$ Optimal respectively. From Fig. 2, the D values associated with the optimal solution for $\mathcal{P}_K$ are notably

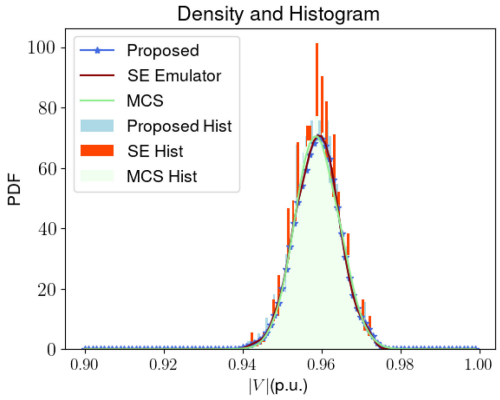


Fig. 3. Comparison of distributions for 123-87-B system.

TABLE IV
COMPARATIVE STUDY OF KLD

DN System	PLF Methods	D_{KL}
33-15	Proposed	0.00863
	SE Emulator	0.1037
37-21-C	Proposed	0.00596
	SE Emulator	0.00629
123-87-B	Proposed	0.0083
	SE Emulator	0.00981

lower than 1 for multiple buses in the DN and this can be attributed to overfitting on $(X_u, f(X_u))$. Moreover, the D values associated with the algorithm presented in [18] are also significantly lower than 1 for multiple buses. This can be attributed to the inability of the SE kernel to extrapolate well [35]. On the other hand, the D values associated with the proposed algorithm are close to 1 for all buses in the DN. Thus, for all buses in IEEE-33 DN the proposed algorithm results in near optimal performance for the testing dataset which is a notable improvement over the state-of-the-art.

In Fig. 3, the histograms and output distributions obtained from the proposed PLF method are plotted against those obtained from the SE Emulator based state-of-the-art and the benchmark MCS method for the 123-87-B test system. For interpretation in practical electrical systems, these distributions are computed over voltage magnitudes ranging from 0.85 p.u to 1 p.u. For, the benchmark MCS method, a sample size of 10000 is utilized. The output distributions for all three methods compared are represented as reconstructed probability distribution function curves. It is clear that the histogram of the proposed technique displays lower variability and closely approximates the benchmark distribution than the SE Emulator based technique. To quantitatively assess the ability of the proposed method to approximate the benchmark distribution (i.e., obtained from the MCS method), in Table IV, the KLDs evaluated against the benchmark distribution are listed for the distributions obtained from the proposed method and SE based technique. It is evident that the proposed technique has lower KLD for all three test systems (i.e., 33-15, 37-21-C and 123-87-B systems) in comparison to the SE Emulator. Thus, it can be concluded that the distribution obtained using the proposed technique closely represents the benchmark distribution.

TABLE V
COMPARATIVE STUDY OF SIMULATION TIME

DN System	PLF Methods	Simulation Time (s)
33-15	Proposed	0.0007
	MCS	0.0029
37-21-C	Proposed	0.0007
	MCS	0.0025
123-87-B	Proposed	0.0021
	MCS	0.0060

In Table V, the average time entailed for inferencing over 600 inputs for both the proposed and the benchmark MCS methods are presented for all three test systems. It is clear from these results that our proposal requires lesser time for inferencing than the benchmark MCS method. This is expected as the computational complexity of our proposal is linear in terms of the number of buses in the system (i.e., $O(n)$). From Eq. (2), the computational complexity of the proposed method is determined by the complexities of the kernel and the inferencing process entailed with the GP emulator. From Eqs. (3) and (4), the kernel utilized by the proposed method is computed from the base kernels and the number of base kernels is the same as the number of features. Since the number of features is linear in terms of the number of buses in the system, the computational complexity of our kernel is $O(n)$ where n is the number of buses in the system. Next, from [49], the time complexity of inferencing with a GP emulator is linear in terms of the size of the training dataset. Since the size of the training dataset is a constant, the complexity of the proposed method in terms of the number of buses can be described by $O(n)$. With the MCS method, the inferencing entails evaluating the load flow relations which according to [47] has a time complexity of $O(n^2)$.

V. CONCLUSION

In this paper, a novel kernel design algorithm is proposed to efficiently approximate the underlying distributions of PLF captured by the training datasets. We show via both theoretical and practical studies that our proposed iterative algorithm converges to the Nash Equilibrium which is also the local optimum. The resulting kernel is also able to approximate the PLF dataset while also generalizing well. In order to demonstrate the contributions of the proposed algorithm with the state-of-the-art, comparative studies are conducted which show that our kernel design algorithm allows for marked improvement in performance. This work can also be leveraged in other applications that utilize GP emulators for learning underlying distributions of datasets.

APPENDIX

To prove $m_{AK_{S^*}}(.)$ can approximate $f(.)$ arbitrarily closely under practical conditions, the following notations are

introduced first. The function obtained through substituting any $x \in \mathbb{R}^{d_i}$ into the second input of $k_{AK_{S^*}}(.,.)$ is denoted as $k_{AK_{S^*}}(.,x)$. This type of function is designated as $k_{AK_{S^*}}(.,.)$ partially evaluated at x. Since $k_{AK_{S^*}}(.,.)$ maps $\mathbb{R}^{d_i} \times \mathbb{R}^{d_i}$ to $\mathbb{R}$, $k_{AK_{S^*}}(.,x)$ maps $\mathbb{R}^{d_i}$ to $\mathbb{R}$. Next, the set of all functions that can be expressed as a weighted sum of $k_{AK_{S^*}}(.,.)$ partially evaluated at emulator inputs in X is represented by $\mathcal{H}^X$. Similarly, the set of all functions that can be expressed as a weighted sum of infinitely many $k_{AK_{S^*}}(.,.)$ where each is partially evaluated at a different emulator input sampled from $\mathbb{R}^{d_i}$ is denoted by $\lim_{n_X \to \infty} \mathcal{H}^X$. That is:

$$\mathcal{H}^X = \left\{ q(.) | q(.) = \sum_i^{n_X} c_i k_{AK_{S^*}}\left(., x^i\right), x^i \in X, c_i \in \mathbb{R} \right\}$$

$$\lim_{n_X \to \infty} = \left\{ q(.) | q(.) = \lim_{n_X \to \infty} \sum_i^{n_X} c_i k_{AK_{S^*}}\left(., x^i\right), x^i \in X, c_i \in \mathbb{R} \right\}$$

The set of non-negative real numbers and the set of positive real numbers are denoted as $\mathbb{R}_{\geq 0}$ and $\mathbb{R}_{>0}$ respectively. A detailed explanation of the mathematical terms and concepts used in the proof of this theorem (e.g., Borel measure) is provided next.

From [48], the sigma-algebra $\mathcal{A}$ of X is a family of subsets of X such that X itself as well as the unions, intersections and complements of elements of $\mathcal{A}$ are in $\mathcal{A}$. That is:

- If $A \in \mathcal{A}$, then $X \setminus A \in \mathcal{A}$.
- If $A, B \in \mathcal{A}$, then $A \cup B \in \mathcal{A}$.
- If $A, B \in \mathcal{A}$, then $A \cap B \in \mathcal{A}$.
- $X \in \mathcal{A}$.

Next, the Borel measure $\hat{\mu}$ on the set X is a function that maps the Borel sigma-algebra of X to $\mathcal{R}_{\geq 0}$, whereas the Borel sigma-algebra for X is the intersection of all sigma-algebras containing the open subsets of X. Then, a subset U of the n-dimensional real space $\mathcal{R}^n$ is open if for every point x in U there exists $\epsilon \in \mathcal{R}_{>0}$ such that $y \in \mathcal{R}^n$ is in U if and only if $\|x - y\| < \epsilon$. Moreover, U is a closed set if its complement is an open set. Furthermore, an open neighborhood O_x of a point x in X is an open subset of X such that there exist $V \in X$ that satisfies $V \in O_x, x \in V$. Finally, the support of a measure $\hat{\mu}$ defined on X is defined as the largest closed subset of X for which the measure for every open neighborhood at every point in the subset is positive.

As such, if $k_{AK_{S^*}}(.,.)$ can be rewritten as the equivalent function $h_{AK_{S^*}}(t)$ where $t = \|x - x'\|^2$, then from [38] there must be a Borel measure $\hat{\mu}_{AK_{S^*}}(\sigma)$ defined on $\mathbb{R}_{\geq 0}$ (i.e., $\sigma \in \mathbb{R}_{\geq 0}$) such that for all $t \in \mathbb{R}_{\geq 0}$, $h_{AK_{S^*}}(t)$ can be expressed in the form of the following Lebesgue integral:

$$h_{AK_{S^*}}(t) = \int_{\mathbb{R}_{\geq 0}} e^{-t\sigma} d\hat{\mu}_{AK_{S^*}}(\sigma)$$

According to [38], for every non-constant $h_{AK_{S^*}}(t)$ the support of $\hat{\mu}_{AK_{S^*}}(\sigma)$ must be contained in $\mathbb{R}_{>0}$. From Theorem 17 in the same reference, if the support of $\hat{\mu}_{AK_{S^*}}(\sigma)$ is contained in $\mathbb{R}_{>0}$ and $d_o = 1$ then there must exist $q^* \in \lim_{n_X \to \infty}$ that can approximate $f(.)$ arbitrarily closely. Thus, there must exist such $q^*(.)$ if $k_{AK_{S^*}}(.,.)$ is not constant. For the dataset $(X, f(X))$, we can write $\|f(x^i) - q^*(x^i)\|^2 \leq \frac{\epsilon}{2^i}$ for every $\epsilon \in \mathbb{R}_{>0}$ without loss of generality since $q^*(.)$ approximates $f(.)$ arbitrarily closely. Thus, the summation of all $\|f(x^i) - q^*(x^i)\|^2$ as $n_X \to \infty$ is upper bounded by every $\epsilon \in \mathbb{R}_{>0}$:

$$\lim_{n_X \to \infty} \sum_{i=1}^{n_X} \|f(x^i) - q^*(x^i)\|^2 \leq \lim_{n_X \to \infty} \sum_{i=1}^{n_X} \frac{\epsilon}{2^i} = \epsilon$$

From the Representer Theorem detailed in [39], the predictive posterior mean $m_{AK_{S^*}}(.)$ for the zero-mean GP $\mathcal{GP}(0, k_{AK_{S^*}}(.,.))$ trained on $(X, f(X))$ satisfies $m_{AK_{S^*}}(.) \in \mathcal{H}^X$ and is the unique solution to the problem:

$$\min_{q \in \mathcal{H}^X} \sum_{i=1}^{n_X} \left\| f(x^i) - q(x^i) \right\|^2$$

From the previous derivations, there exists $q^*(.) \in \lim_{n_X \to \infty}$ such that $\sum_{i=1}^{n_X} \|f(x^i) - q^*(x^i)\|^2$ is finite. Thus, as $n_X \to \infty$ the posterior mean $m_{AK_{S^*}}(.)$ trained on $(X, f(X))$ satisfies $m_{AK_{S^*}}(.) \in \lim_{n_X \to \infty}$ and is the unique solution to the problem:

$$\min_{q \in \lim_{n_X \to \infty} \mathcal{H}^X} \lim_{n_X \to \infty} \sum_{i=1}^{n_X} \left\| f(x^i) - q(x^i) \right\|^2$$

As $m_{AK_{S^*}}(.)$ is the unique solution to the above problem, we further have:

$$\lim_{n_X \to \infty} \sum_{i=1}^{n_X} \left\| f(x^i) - m_{AK_{S^*}}(x^i) \right\|^2 \leq \lim_{n_X \to \infty} \sum_{i=1}^{n_X} \left\| f(x^i) - q^*(x^i) \right\|^2 \leq \epsilon$$

For all $\epsilon \in \mathbb{R}_{>0}$. From the above relation, we have $\|f(x) - m_{AK_{S^*}}(x)\|^2 \leq \epsilon$ for every $x \in \mathbb{R}^{d_i}, \epsilon \in \mathbb{R}_{>0}$. Thus, the posterior mean $m_{AK_{S^*}}(.)$ produced by Algorithm 1 can approximate $f(.)$ arbitrarily closely.

REFERENCES

[1] "Smart grid system report," U.S. Dept. Energy, Washington, DC, USA, Rep., 2018.

[2] B. Borkowska, "Probabilistic load flow," *IEEE Trans. Power App. Syst.*, vol. PAS-93, no. 3, pp. 752–759, May 1974.

[3] Y. Yang, Z. Yang, J. Yu, B. Zhang, Y. Zhang, and H. Yu, "Fast calculation of probabilistic power flow: A model-based deep learning approach," *IEEE Trans. Smart Grid*, vol. 11, no. 3, pp. 2235–2244, May 2020.

[4] C. D. Zuluaga and M. A. Álvarez, "Bayesian probabilistic power flow analysis using Jacobian approximate Bayesian computation," *IEEE Trans. Power Syst.*, vol. 33, no. 5, pp. 5217–5225, Sep. 2018.

[5] H. Yu, C. Y. Chung, K. P. Wong, H. W. Lee, and J. H. Zhang, "Probabilistic load flow evaluation with hybrid Latin hypercube sampling and Cholesky decomposition," *IEEE Trans. Power Syst.*, vol. 24, no. 2, pp. 661–667, May 2009.

[6] J. Huang, Y. Xue, Z. Y. Dong, and K. P. Wong, "An adaptive importance sampling method for probabilistic optimal power flow," in *Proc. IEEE Power Energy Soc. Gen. Meeting*, Detroit, MI, USA, 2011, pp. 1–6.

[7] M. E. Nassar, A. A. Hamad, M. M. A. Salama, and E. F. El-Saadany, "A novel load flow algorithm for islanded AC/DC hybrid microgrids," *IEEE Trans. Smart Grid*, vol. 10, no. 2, pp. 1553–1566, Mar. 2019.

[8] C. Liu, K. Sun, B. Wang, and W. Yu, "Probabilistic power flow analysis using multidimensional holomorphic embedding and generalized cumulants," *IEEE Trans. Power Syst.*, vol. 33, no. 6, pp. 7132–7142, Nov. 2018.

[9] M. Nijhuis, M. Gibescu, and S. Cobben, "Gaussian mixture based probabilistic load flow for LV-network planning," *IEEE Trans. Power Syst.*, vol. 32, no. 4, pp. 2878–2886, Jul. 2017.

[10] R. Allan and R. Billinton, "Probabilistic assessment of power systems," *Proc. IEEE*, vol. 88, no. 2, pp. 140–162, Feb. 2000.

[11] H. Sheng and X. Wang, "Probabilistic power flow calculation using non-intrusive low-rank approximation method," *IEEE Trans. Power Syst.*, vol. 34, no. 4, pp. 3014–3025, Jul. 2019.
[12] C.-C. Yang and Y.-Y. Hsu, "Estimation of line flows and bus voltages using decision trees," *IEEE Trans. Power Syst.*, vol. 9, no. 3, pp. 1569–1574, Aug. 1994.
[13] P. Amid and C. Crawford, "A cumulant-tensor based probabilistic load flow method," *IEEE Trans. Power Syst.*, vol. 33, no. 5, pp. 5648–5656, Sep. 2018.
[14] G. Gruosso, P. Maffezzoni, Z. Zhang, and L. Daniel, "Probabilistic load flow methodology for distribution networks including loads uncertainty," *Int. J. Electr. Power Energy Syst.*, vol. 106, pp. 392–400, Mar. 2019.
[15] H. Nosratabadi, M. Mohammadi, and A. Kargarian, "Nonparametric probabilistic unbalanced power flow with adaptive kernel density estimator," *IEEE Trans. Smart Grid*, vol. 10, no. 3, pp. 3292–3300, May 2019.
[16] M. Aien, M. Fotuhi-Firuzabad, and F. Aminifar, "Probabilistic load flow in correlated uncertain environment using unscented transformation," *IEEE Trans. Power Syst.*, vol. 27, no. 4, pp. 2233–2241, Nov. 2012.
[17] Z. Ren, W. Li, R. Billinton, and W. Yan, "Probabilistic power flow analysis based on the stochastic response surface method," *IEEE Trans. Power Syst.*, vol. 31, no. 3, pp. 2307–2315, May 2016.
[18] Y. Xu, Z. Hu, L. Mili, M. Korkali, and X. Chen, "Probabilistic power flow based on a Gaussian process emulator," *IEEE Trans. Power Syst.*, vol. 35, no. 4, pp. 3278–3281, Jul. 2020.
[19] J. Tang, F. Ni, F. Ponci, and A. Monti, "Dimension-adaptive sparse grid interpolation for uncertainty quantification in modern power systems: Probabilistic power flow," *IEEE Trans. Power Syst.*, vol. 31, no. 2, pp. 907–919, Mar. 2016.
[20] Y. Zhang and J. Wang, "Towards highly efficient state estimation with nonlinear measurements in distribution systems," *IEEE Trans. Power Syst.*, vol. 35, no. 3, pp. 2471–2474, May 2020.
[21] Y. Bengio, O. Delalleau, and N. Le Roux, "The curse of highly variable functions for local kernel machines," in *Advances in Neural Information Processing Systems*, vol. 18. Cambridge, MA USA: MIT Press, 2006, pp. 107–114.
[22] D. Duvenaud, H. Nickisch, and C. E. Rasmussen, "Additive Gaussian processes," in *Proc. Int. Conf. Adv. Neural Inf. Process. Syst.*, vol. 24. Granada, Spain, 2011, pp. 226–234.
[23] *Distribution Automation Handbook*, ABB Group, Zürich, Switzerland, 2011.
[24] K. P. Schneider *et al.*, "Analytic considerations and design basis for the IEEE distribution test feeders," *IEEE Trans. Power Syst.*, vol. 33, no. 3, pp. 3181–3188, May 2018.
[25] L. Gan and S. H. Low, "Convex relaxations and linear approximation for optimal power flow in multiphase radial networks," in *Proc. Power Syst. Comput. Conf.*, 2014, pp. 1–9.
[26] M. Alvarez and N. D. Lawrence, "Sparse convolved Gaussian processes for multi-output regression," in *Advances in Neural Information Processing Systems (NeuralPS) 21*. La Jolla, CA, USA: Neural Inf. Process. Syst., 2009, pp. 57–64.
[27] B. Rotthier, T. V. Maerhem, P. Blockx, P. V. Bossche, and J. Cappelle, "Home charging of electric vehicles in Belgium," in *Proc. 27th Electr. Veh. Symp.*, Barcelona, Spain, Nov. 2013, pp. 1–6.
[28] E. Yao, P. Samadi, V. W. S. Wong, and R. Schober, "Residential demand side management under high penetration of rooftop photovoltaic units," *IEEE Trans. Smart Grid*, vol. 7, no. 3, pp. 1597–1608, May 2016.
[29] G. Li and X.-P. Zhang, "Modeling of plug-in hybrid electric vehicle charging demand in probabilistic power flow calculations," *IEEE Trans. Smart Grid*, vol. 3, no. 1, pp. 492–499, Mar. 2012.
[30] J. W. Shim, G. Verbič, and K. Hur, "Stochastic Eigen-analysis of electric power system with high renewable penetration: Impact of changing inertia on oscillatory modes," *IEEE Trans. Power Syst.*, vol. 35, no. 6, pp. 4655–4665, Nov. 2020.
[31] Z. Liang, H. Chen, X. Wang. S. Chen, and C. Zhang, "Risk-based uncertainty set optimization method for energy management of hybrid AC/DC microgrids with uncertain renewable generation," *IEEE Trans. Smart Grid*, vol. 11, no. 2, pp. 1526–1542, Mar. 2020.
[32] P. T. Staats, W. M. Grady, A. Arapostathis, and R. S. Thallam, "A statistical analysis of the effect of electric vehicle battery charging on distribution system harmonic voltages," *IEEE Trans. Power Del.*, vol. 13, no. 2, pp. 640–646, Apr. 1998.
[33] J. R. Pillai and B. Bak-Jensen, "Impacts of electric vehicle loads on power distribution systems," in *Proc. IEEE Veh. Power Propulsion Conf.*, Lille, France, Sep. 2010, pp. 1–6.
[34] R. R. Richardson, M. A. Osborne, and D. A. Howey "Gaussian process regression for forecasting battery state of health," *J. Power Sour.*, vol. 357, pp. 209–219, Jul. 2017.
[35] D. K. Duvenaud, "Automatic model construction with Gaussian processes," Ph.D. dissertation, Dept. Doctor Philos., Univ. Cambridge, Cambridge, U.K., 2014.
[36] B. Colson, P. Marcotte, and G. Savard, "An overview of bilevel optimization," *Ann. Oper. Res.*, vol. 153, pp. 235–256, Apr. 2007.
[37] D. Monderer and L. Shapley, "Potential games," *Games Econ. Behav.*, vol. 14, no. 1, pp. 124–143, 1996.
[38] C. A. Micchelli, Y. Xu, and H. Zhang, "Universal kernels," *J. Mach. Learn. Res.*, vol. 7, pp. 2651–2667, Dec. 2006.
[39] B. Schölkopf, R. Herbrich, and A. J. Smola, "A generalized representer theorem," in *Proc. Conf. Comput. Learn. Theory*, 2001, pp. 416–426.
[40] E. Bradford and L. Imsland, "Combining Gaussian Processes and polynomial chaos expansions for stochastic nonlinear model predictive control," 2021, *arXiv:2103.05441*.
[41] B. Sudret, "Polynomial chaos expansions and stochastic finite element methods," in *Risk and Reliability in Geotechnical Engineering*. Boca Raton, FL, USA: CRC Press, 2014, pp. 265–300.
[42] R. C. Dugan, *Reference Guide: The Open Distribution System Simulator (OpenDSS)*, EPRI, Washington, DC, USA, 2016.
[43] P. Pareek and H. D. Nguyen, "Gaussian Process Learning-Based Probabilistic Optimal Power Flow," *IEEE Trans. Smart Grid*, vol. 36, no. 1, pp. 541–544, Jan. 2021.
[44] J. Lever, M. Krzywinski, and N. Altman, "Model selection and overfitting," *Nat. Methods*, vol. 13, pp. 703–704, Aug. 2016.
[45] A. G. D. G. Matthews *et al.*, "GPflow: A Gaussian process library using tensorflow," *J. Mach. Learn. Res.*, vol. 18, pp. 1–6, Jan. 2017.
[46] K. P. Burnham and D. R. Anderson, *Model Selection and MultiModel Inference*, 2nd ed. New York, NY, USA: Springer, 2002, p. 51.
[47] M. Marin, D. Defour, and F. Milano, "Power flow analysis under uncertainty using symmetric fuzzy arithmetic," in *Proc. IEEE PES Gen. Meeting*, 2014, pp. 1–5.
[48] S. Axler, *Measure, Integration & Real Analysis* (Graduate Texts in Mathematics). Cham, Switzerland: Springer, 2020.
[49] H. Liu, Y.-S. Ong, X. Shen, and J. Cai, "When Gaussian process meets big data: A review of scalable GPs," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 11, pp. 4405–4423, Nov. 2020.

Jingyuan Liu (Student Member, IEEE) received the B.Sc. degree in aerospace engineering from the University of Illinois Urbana–Champaign, Champaign, IL, USA, in 2015, and the M.Sc. degree in aerospace engineering from the Purdue University, West Lafayette, IN, USA, in 2017. He is currently pursuing the Ph.D. degree with the Department of Electrical Engineering and Computer Science, York University, Toronto, ON, Canada. His current research interests include optimization and control of power systems and smart grid security, with a particular focus on the application of game theoretic and convex optimization techniques to distribution network reconfiguration and optimal power flow problems.

Pirathayini Srikantha (Member, IEEE) received the B.A.Sc. degree in systems design engineering and the M.A.Sc. degree in electrical and computer engineering from the University of Waterloo in 2009 and 2013, respectively, and the Ph.D. degree from The Edward S. Rogers Sr. Department of Electrical and Computer Engineering, University of Toronto in 2017. She is a Canada Research Chair (Tier 2) and an Assistant Professor with the Department of Electrical Engineering and Computer Science, York University. She is a certified Professional Engineer (P.Eng.) in Ontario. Her work has been published in premier smart grid journal and conference venues. Her main research interests are in the areas of large-scale optimization and distributed control for enabling adaptive, sustainable and resilient power grid operations. Her research efforts have received recognitions that include the Best Paper Award (IEEE Smart Grid Communications) and the Runner-Up Best Poster Award (ACM Women in Computing). She is currently serving as an Associate Editor for the IEEE TRANSACTIONS ON SMART GRID journal.